

Reproducibility Report for the Paper: “gHyPart: GPU-friendly End-to-End Hypergraph Partitioner”

Matteo Federico

Department of Civil and Computer Engineering
University of Rome “Tor Vergata”
Rome, Italy
matteo.federico@uniroma2.it

Simone Bauco

Department of Civil and Computer Engineering
University of Rome “Tor Vergata”
Rome, Italy
simone.bauco@uniroma2.it

Simone Staccone

Department of Civil and Computer Engineering
University of Rome “Tor Vergata”
Rome, Italy
simone.staccone@uniroma2.it

Abstract—The paper “gHyPart: GPU-friendly End-to-End Hypergraph Partitioner” explores the potential of GPUs in the field of hypergraph partitioning, introducing gHyPart, the first end-to-end deterministic hypergraph partitioner on the GPU.

This report examines the reproducibility of the software artifacts associated with GHYPART across different GPU architectures, with particular attention to performance and portability.

I. INTRODUCTION

This report evaluates the reproducibility of GHYPART, an end-to-end GPU-based hypergraph partitioner [1], using the authors’ software artifact [2].

Hypergraph partitioning is vital in HPC, VLSI design, SpMV optimization, and large-scale data analysis. Yet most partitioners target CPUs, leaving the massive parallelism and high memory bandwidth of GPUs underused. GHYPART addresses this by providing, to the best of the authors’ knowledge, the first GPU-native implementation of the full multilevel hypergraph partitioning workflow.

The system follows the standard multilevel scheme of coarsening, initial partitioning, and refinement. To run efficiently on GPUs, the authors redesign data structures and kernels to improve load balance, reduce overhead, and shorten critical paths. For each major GPU kernel, several parallelization strategies are available; a decision-tree-based selector chooses the best variant per instance, since no single strategy is optimal for all hypergraphs.

GHYPART is built on GHYPART-B, a baseline GPU port of the CPU-based BiPart algorithm. Analysis of this baseline identifies three main issues: thread underutilization, long critical paths, and suboptimal workload in hypergraph construction. GHYPART tackles these via optimized kernels and the decision-tree selector.

In the original publication, the authors evaluate GHYPART on a benchmark comprising 500 real-world hypergraphs and report substantial performance improvements relative to GHYPART-B and state-of-the-art CPU-based alternatives.

This reproducibility study investigates how far paper results can be independently confirmed. It assesses the completeness and quality of the artifact, the clarity of its documentation, and its portability across four GPU architectures and multiple

toolchains, with particular focus on how closely reproduced performance matches the published measurements.

II. GOALS AND MOTIVATION

Reproducibility is a core requirement of scientific methodology, enabling independent verification of results and supporting the cumulative advancement of knowledge. In computer science, however, it is often difficult to ensure due to complex software ecosystems, long dependency chains, and heterogeneous hardware platforms. These issues can hinder reliable replication of published results, lowering confidence in scientific claims and limiting their practical impact.

Reproducibility-focused evaluation is especially important for highly optimized tools such as GPU-based algorithms, whose behavior depends not only on conceptual correctness but also on low-level architectural features. Here, access to a working software artifact lets researchers treat the implementation as a black box: understanding its intended functionality is sufficient, without needing full insight into all implementation details.

This report systematically assesses the reproducibility of GHYPART, aiming to examine the reliability, portability, and experimental robustness of the provided software artifact. By reproducing experiments across multiple GPU architectures, we evaluate how well the original results can be independently confirmed and identify factors that affect reproducibility in GPU-centric research.

The contributions of this study are:

- An evaluation of the hardware architectures on which GHYPART can be compiled and executed.
- A documentation of the reproducibility challenges encountered, together with possible mitigation strategies.
- An analysis of how the observed experimental results relate to the hardware platforms used.

Overall, this report offers a rigorous, transparent account of the reproducibility of GHYPART, contributing to efforts to improve the reliability and longevity of software artifacts in computational research.

III. QUALITY OF THE ARTIFACT

To systematically evaluate the quality of a software artifact, it is helpful to refer to the classification of software artifacts

The authors acknowledge CloudLab and IT-Systems for letting us use different NVIDIA GPUs

Component	Authors	Report
GPU	NVIDIA RTX 3090 NVIDIA RTX 4060 Ti	NVIDIA GTX 1060 NVIDIA RTX 2070 NVIDIA RTX 4090 Quadro RTX 5000
CPU	Intel Xeon 4214R Intel Core i9-13900K Intel Xeon Platinum 8358P	Intel Xeon E5-2683 Intel Core i7-6700

TABLE I: Hardware platforms used in the original experiments (Authors) and in our reproduction experiments.

provided by the ACM Artifact Evaluation taxonomy [3]. The ACM badge system distinguishes between different levels of reproducibility:

- **Artifacts Available:** The authors have made the software artifact publicly accessible in a permanent, citable repository.
- **Artifacts Evaluated - Functional:** The provided artifacts are documented, consistent, complete and exercisable.
- **Artifacts Evaluated - Reusable:** the artifacts have all the qualities of the Artifacts Evaluated – Functional level, but, in addition, they are very carefully documented and well-structured to the extent that reuse and repurposing is facilitated.
- **Results Reproduced:** The main results of the paper have been obtained by independent reviewers using artifacts provided by the author.

Each badge reflects an increasing degree of confidence in the reliability, completeness, and reusability of the provided materials.

Although the GHYPART repository clearly meets the criteria for the Artifact Available badge, as the full codebase is publicly accessible, achieving higher reproducibility levels requires closer examination. Specifically, assessing eligibility for the remaining badges demands a systematic evaluation of buildability, portability, documentation quality, and the degree to which the paper’s experimental results can be independently replicated.

This report therefore evaluates whether the GHYPART software artifact meets the criteria for the Functional, Reusable, and Results Reproduced badges and identifies steps to further enhance its robustness and overall quality.

IV. ORIGINAL ARTIFACT QUALITY ASSESSMENT

In the case of GHYPART, the authors provide all relevant software artifacts within a single GitHub repository. The primary documentation is contained in the `README.md` file and in the accompanying publication. The `README.md` file offers a concise overview of the build and execution procedures for the three systems evaluated in the paper, `gHyPart`, `BiPart`, and `Mt-KaHyPar`. Compilation relies on CMake, the GNU C++ compiler (`g++`), and the NVIDIA CUDA compiler (`nvcc`).

Additionally, a small number of dependencies must be installed to enable successful compilation and execution. These dependencies are limited to widely available C++ libraries,

such as Boost, and a few Python modules used by the provided evaluation scripts.

Overall, the required dependencies are straightforward to obtain and do not present a significant barrier to reproducibility.

A. Code Structure

The repository contains the full source code for all evaluated partitioners, organized into modular components corresponding to GPU, multi-threaded CPU, and sequential CPU implementations. The `gHyPart` implementations comprise the main GPU kernels, data structure definitions, and coarsening/partitioning/refinement procedures. The CPU-based baselines `BiPart` and `Mt-KaHyPar` are embedded in dedicated subdirectories and can be built through the same CMake workflow, while `HMETIS` is included as a precompiled static library.

In addition to the core implementation, the repository includes two families of Python scripts: (i) scripts used to process and analyze hypergraphs, and (ii) scripts to run evaluation pipelines and generate figures comparable to those presented in the original publication.

B. Hardware Platform

Experiments were executed on multiple GPU and CPU systems to evaluate the portability, sensitivity to architectural differences, and overall robustness of the artifact. The hardware platforms originally tested by the authors included NVIDIA RTX 3090 and RTX 4060 Ti GPUs, and Intel Xeon 4214R, Intel Core i9-13900K, and Intel Xeon Platinum 8358P CPUs. In addition, this study extended the evaluation to include NVIDIA GTX 1060, RTX 2070, RTX 4090, and Quadro RTX 5000 GPUs, as well as Intel Xeon CPU E5-2683 and Intel Core i7-6700 CPUs. Hardware overview is reported in Table I.

C. Software and Toolchain Versions

According to the authors and as verified within the repository, the software artifact was compiled using CMake 3.11, the GNU C++ compiler GCC 9.4, and the NVIDIA CUDA compiler (`nvcc`) version 12.0, with compilation flags set to `-O3 -arch=sm_86`.

D. Dataset

The original work uses two datasets: one to train the decision-tree strategy selector and one to evaluate performance. The training dataset contains synthetic hypergraphs, generated because few public real-world hypergraphs with diverse structures exist.

The evaluation dataset includes 500 real-world hypergraphs from prior hypergraph partitioning studies, all stored in `hMetis` format. Hypergraphs are taken from the `uiteSparse Matrix Collection`, the `ISPD98 VLSI Circuit Benchmark Suite`, the `DAC 2012 Routability-Driven Placement Contest`, and the `2014 SAT Competition`. The original paper reports detailed statistics, including percentile distributions of key hypergraph features.

The real-world dataset is available via two download links (SharePoint and Zenodo). The decision-tree training dataset is only described in the paper and has no explicit download link.

V. REPRODUCIBILITY

To ensure experimental reproducibility and consistent results, we focused on the build process. The source code alone does not guarantee successful compilation and execution, as different compiler and build tool versions can introduce non-trivial delays in software deployment. Moreover, since GHYPART runs on a GPU, the driver handling host-device communication must be compatible with the installed CUDA Toolkit, or compilation and runtime conflicts may occur.

Every change in the original repository is reported in a fork on https://github.com/HiPeac-TorVergata/gHyPart_TACO

A. Build Process

The initial build of GHYPART was not straightforward and required several adjustments to successfully compile the code. The primary challenge stemmed from the GPU architecture specification within the CMake build scripts: the code was originally configured for newer GPU architectures (Ampere, `sm_86`) and failed to compile on older GPUs due to the use of the `-arch` flag in the `nvcc` compiler. This required modifying the CMake scripts to target the appropriate architecture for the GPUs available during reproduction. In practice, we relied on the output of the `nvidia-smi` command to determine the correct compute capability to use during compilation. When this automatic detection is not feasible, such as in HPC environments relying on a `module load` toolchain, GPU information may not be directly accessible during configuration and therefore the build system is configured to fall back to a default architecture of `sm_75`. This heuristic value proved to be sufficiently robust, as it enabled successful compilation across all experimental platforms considered in this report. The update of the CMake script also required upgrading the minimum CMake version to 3.18 from the original 3.11, in order to use newer CUDA detection mechanism.

Another significant issue emerged at runtime and appeared to be indirectly caused by the build configuration. Specifically, certain combinations of the GNU compiler (`gcc`) and the NVIDIA CUDA compiler (`nvcc`) resulted in a segmentation fault during execution. In these cases, the error manifested as a failure which is unable to correctly detect or initialize the available GPU with the following error message:

```
terminate called after throwing an instance of
'thrust::THRUST_200700_750_NS::system::system_error'
what(): parallel_for failed: cudaErrorInvalidDevice:
invalid device ordinal
```

This behavior is reproducible across several mismatched compiler-toolkit configurations, suggesting a compatibility problem between the selected host compiler and the CUDA runtime.

Although this issue has not been fully resolved, all software configurations that successfully compiled and executed the code, each associated with its respective hardware platform, are reported in Table II. This provides a complete overview of

GPU	CUDA Toolkit	GCC Version	NVIDIA Driver
GTX 1060	12.0	9.4	525.60.13
RTX 2070	12.0	9.4	550.163.61
Quadro RTX 5000	12.5	13.3	570.172.08
RTX 4090	12.4	13.3	550.54.14
RTX 3090 (authors)	12.0	9.4	Not Available
RTX 4060 Ti (authors)	12.0	9.4	Not Available

TABLE II: Software configurations successfully used during reproduction.

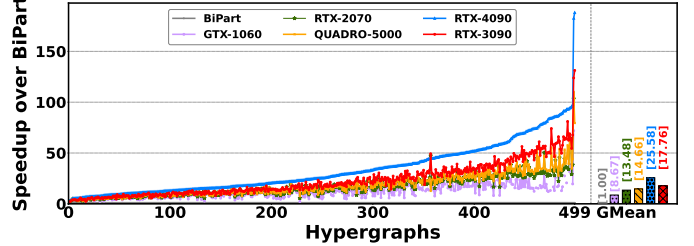


Fig. 1: GHYPART’s end-to-end speedup over BiPart across different GPUs

the compiler, toolkit, and driver versions that yielded correct execution during our reproducibility study.

B. Experiments Execution

The second step toward reproducing the published results consists of executing the Python scripts provided in the repository, which are responsible for running the experiments and generating the corresponding plots. However, the repository lacks documentation explaining the purpose and functionality of these scripts. Moreover, the code contains substantial amounts of unused or obsolete segments, is almost entirely devoid of comments, and includes hard-coded file paths within several scripts, making it difficult to understand the intended workflow.

Despite these limitations, the authors responded promptly to our inquiries via email, providing essential guidance and clarifying the procedure required to reproduce the results of both GHYPART-B and GHYPART-O. Their support was crucial in establishing the appropriate sequence of procedures required to execute the experiments successfully, despite the need for certain modifications to obtain the figures presented in this report.

The first issue encountered is that executing GHYPART on older GPU architectures may result in errors attributable to the limited capacity of the GPU memory, leading to out-of-memory conditions during the coarsening phase. While this is not unexpected for devices with reduced RAM, the problem is that such failures are not detected or handled by the software. As a result, the execution time recorded in the final `.csv` files is not flagged as an error but is instead treated as valid data and included in the plotted figures. This behavior was observed on all tested GPUs except the RTX 4090, indicating that it is not an isolated case. Therefore, implementing proper error

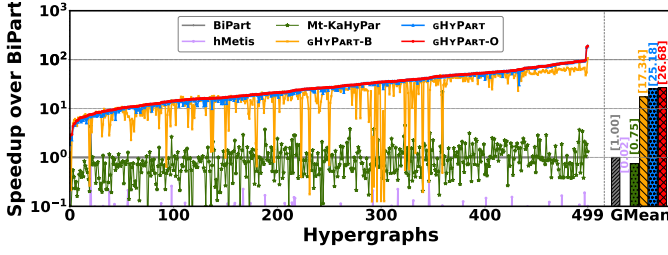


Fig. 2: Speedup of all considered partitioning methods over BiPart across the evaluated hypergraphs

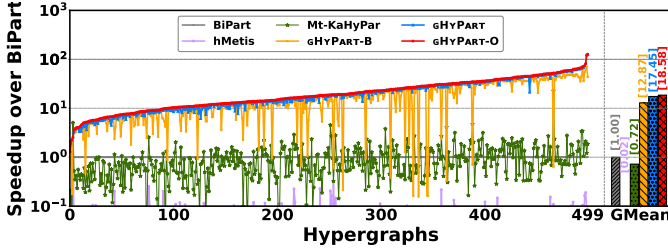


Fig. 3: Original speedup of all considered partitioning methods over BiPart across the evaluated hypergraphs (Figure 11)

handling is essential to avoid reporting synthetic or misleading timing results.

Secondly, although figure plotting scripts are provided, they rely also on a set of .csv files that cannot be regenerated from the available code. As a result, while the figures presented in the authors’ original publication can be reproduced using the .csv files provided in the associated repository, it is not possible to generate the same figures when relying on the data acquired during the present reproduction campaign. In other words, the original results can be regenerated, but the workflow does not support re-execution of the experiments to obtain updated figures from newly gathered data. Instead, additional effort was required to reconstruct the plots by generating new data.

Lastly, a more systematic and modular workflow would significantly improve the reusability of the artifact. In particular, the adoption of an integrated workflow, in which a master script orchestrates the complete procedure (including recompilation of the project, execution of all experiments, and aggregation of the resulting data) would promote methodological rigor and ensure consistency across experimental runs. Subsequent scripts could then operate solely on the generated data, making the workflow cleaner, more maintainable, and easier to reproduce.

C. Performance comparison

Following the authors’ methodology, we evaluate GHYPART’s performance by measuring the end-to-end speedup over BiPart in the 2-way partitioning scenario across 500 hypergraphs. The results, reported in Figure 1, illustrate GHYPART’s speedup on all the considered GPUs, where missing

points correspond to runs that failed due to out-of-memory errors.

Overall, we can observe results in line with those presented in the paper. The authors report a $17.76\times$ average speedup on the NVIDIA RTX 3090 GPU. In our experiments, we observe average speedups of $8.67\times$, $13.48\times$, $14.66\times$, and $25.58\times$ on the NVIDIA GTX 1060, NVIDIA RTX 2070, NVIDIA Quadro RTX 5000 and NVIDIA RTX 4090 GPUs, respectively.

The observed speedup correlates with the underlying hardware characteristics of each GPU, with higher-end GPUs (e.g., having a larger number of Streaming Multiprocessors) achieving superior performance.

Results in Figure 2 show the speedup over BiPart achieved by all partitioners evaluated in this study on the NVIDIA RTX 4090, while Figure 3 shows the same original results. In particular, the speedups obtained by GHYPART relative to the CPU-based benchmarks are consistent with those reported in the original paper: $25.18/0.75 = 33.57\times$ over Mt-KaHyPar, $25.18\times$ over BiPart and $25.18/0.02 = 1259\times$ over hMetis. These results, obtained on a GPU with superior hardware capabilities compared to the one used in the original experiments, are consistent with the expected performance improvements.

Additionally, GHYPART yields an additional speedup of about $8\times$ relative to the GPU baseline GHYPART-B, showing a more substantial performance gain rather than the one reported in the paper (about $4.5\times$), while still exhibiting the same fluctuating trend for GHYPART-B observed in the original results. Lastly, in line with the findings of the paper, GHYPART closely approaches GHYPART-O.

Figure 4 evaluates the efficacy of GHYPART on the 100 most computationally-intensive hypergraphs, showing results consistent with the original ones presented in Figure 5. The comparison between the plots indicates that the speedup of GHYPART over GHYPART-B is slightly reduced. This is due to the increased computational capability of the RTX 4090 compared to the RTX 3090 (approximately 6,000 additional CUDA cores) which boosts the baseline performance and consequently diminishes the relative impact of the parallelization optimizations in GHYPART.

In conclusion, we assess GHYPART’s performance on k -way partitioning on the NVIDIA RTX 4090. Figure 6 illustrates the average speedup of GHYPART over BiPart for $k = 2, 3, 4$ partitions. The results confirm the decreasing trend in GHYPART’s speedup and the corresponding increase in Mt-KaHyPar’s performance with increasing k , as observed in the original paper in Figure 7.

D. Partitioning Quality Results

While execution time is a relevant concern, the main goal of hypergraph partitioning is the maximization of partitioning quality. The quality of a partitioning method can be quantitatively evaluated by measuring the communication overhead between different partitions.

To compare the partitioning quality of GHYPART against state-of-the-art baselines, the authors introduce a comparative

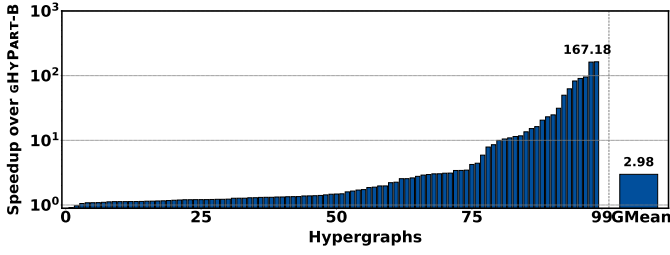


Fig. 4: Speedup of gHyPart over gHyPart-B in the 100 most time-consuming hypergraphs using RTX 4090

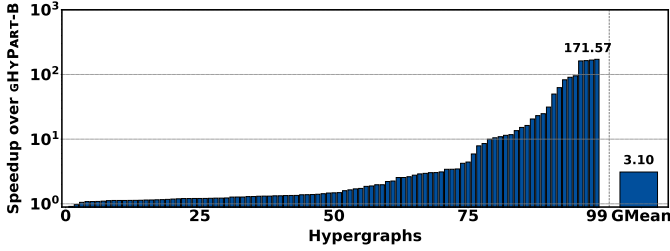


Fig. 5: Original speedup of gHyPart over gHyPart-B in the 100 most time-consuming hypergraphs using RTX 3090 (Figure 12)

metric defined as follows. Given a hypergraph $H = (V, E)$ and two partitioning methods A and B , the relative difference in the number of hyperedge cuts produced by the two methods is computed with respect to the total number of hyperedges: $(cut_A(H) - cut_B(H))/|E|$. The authors classify the results into three categories: the methods are deemed "Comparable" when the ratio's absolute value is lower than 0.5%, while method A is considered "Better" when the ratio is negative, and "Worse" when it is positive.

Table III presents the results obtained by reproducing the experiments in this report alongside the data reported by the authors. In addition to the authors' results, we executed HMETIS without imposing timeouts, excluding only those hypergraphs that resulted in a segmentation fault. Overall, the results are largely consistent with those reported by the authors, except for HMETIS, where the runs that previously timed out yield, in most cases, better results than GHYPART.

Secondly, our objective was to quantify the extent to which one partitioner outperforms or underperforms another. To this end, we constructed a box plot, presented in Figure 8. This plot reports the normalized difference $(cut_A(H) - cut_B(H))/|E|$ for each hypergraph, considering the results for $k = 2, 3, 4$. The results indicate that 2-way partitions are stable and consistent with the tabulated performance, whereas 3-way and 4-way partitions exhibit a significantly worse quality of the results in the worst cases.

Consequently, based on this analysis, Figure 9 examines how performance changes when using the best policies provided by the authors to obtain higher-quality results. The results show that performance varies noticeably, and in some cases significantly, when switching from the standard policy to

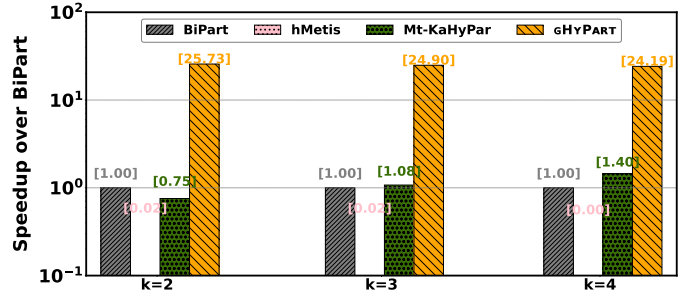


Fig. 6: End-to-end speedup over BiPart for k-way partitioning tasks

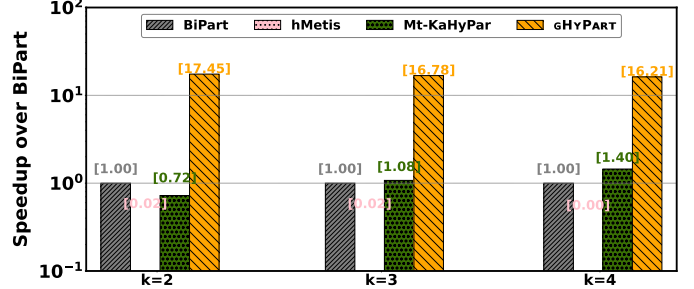


Fig. 7: Original end-to-end speedup over BiPart for k-way partitioning tasks (Figure 18)

the best one. However, no consistent trend emerges: for nearly every value of k , approximately half of the hypergraphs exhibit improved performance, while the other half perform worse.

VI. FUTURE WORK

The analysis conducted in this report highlights several aspects of GHYPART that deserve deeper investigation beyond the scope of the current reproducibility study. The following considerations outline potential avenues for future work aimed at strengthening the comparative understanding of GPU-based hypergraph partitioners and improving the overall quality of the approach.

A first relevant line of investigation would be to compare the results of HYPERG [4] with those of GHYPART, since both approaches address hypergraph partitioning on GPU architectures and employ the ISPD98 circuit benchmark [5] as a common evaluation problem. The analysis of the results reported in both papers reveals that only a single hypergraph, `circuit3`, is present in both experimental datasets. According to the reported results, GHYPART achieves better outcomes on this hypergraph when executed on an RTX 3090, compared to HYPERG on an A600 GPU. Although these are different architectures, they have the same number of CUDA cores. Furthermore, GHYPART demonstrates superior performance and partitioning quality for the 2-way partitioning of this hypergraph. While this observation does not constitute definitive evidence that one solution is universally superior to the other, it highlights the potential value of a more extensive comparative analysis.

Work A vs. Work B	$k = 2$ (Total 494 (385) for hMetis)			$k = 3$ (Total 493 (422) for hMetis)			$k = 4$ (Total 492 (469) for hMetis)		
	% Better	% Comparable	% Worse	% Better	% Comparable	% Worse	% Better	% Comparable	% Worse
gHyPart vs. BiPart	6.4% (6%)	90.8% (91%)	2.8% (3%)	10.8% (11%)	79.2% (80%)	10.0% (9%)	13.2% (13%)	76.8% (78%)	10.0% (9%)
gHyPart vs. Mt-KaHyPar	5.8% (6%)	71.4% (71%)	22.8% (23%)	1.2% (1%)	33.6% (34%)	65.2% (65%)	2.4% (2%)	27.6% (28%)	70% (70%)
gHyPart vs. hMetis	11.5% (18%)	54% (50%)	34.5% (32%)	8 (16)%	24.7% (21%)	67.3% (63%)	7.3 (16)%	22% (23%)	70.7% (61%)

TABLE III: Partitioning quality results obtained in the reproducibility experiments, compared with the results reported in the original paper (shown in parentheses). Note that hMETIS processes only a subset of the evaluated hypergraphs within the specified timeout

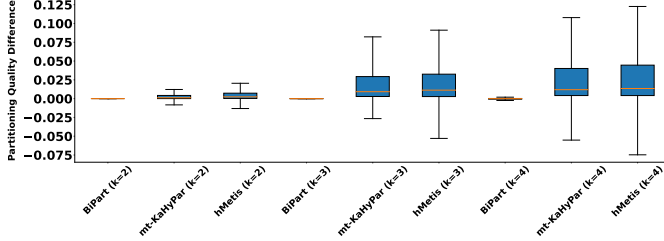


Fig. 8: Partitioning quality quantification results. Outliers are excluded from the plot. Bigger box plot means less compatible results and higher values show worst results of GHYPART

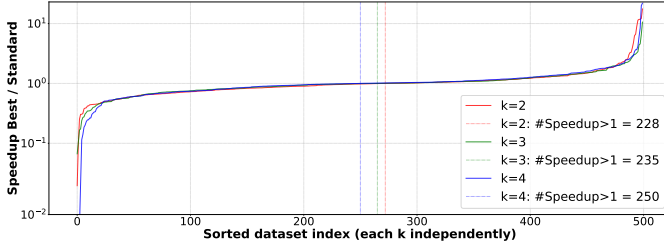


Fig. 9: Standard GHYPART execution compared with policy execution on RTX 4090. Comparison is made on the normalized difference of the partition edge cut cardinality.

A second important consideration concerns the motivation behind the decision tree used to optimize the parallelization strategy. In particular, as observed in Figure 10, some differences in result quality are apparent between GHYPART and GHYPART-B. This suggests that incorporating quality metrics into the optimization process and therefore study how the training of the decision tree used to optimize the parallelization strategy could impact on the quality of the partition.

VII. CONCLUSION

The primary contribution of this report is the rigorous reproduction and independent validation of the results presented for GHYPART. Our findings confirm not only the novelty of the method, positioning it alongside HYPERG as one of the first GPU-accelerated hypergraph partitioners, but also its strong performance characteristics and the overall robustness of its results.

Taking into account the limitations encountered during the reproducibility process, the GHYPART software artifact can currently be classified only as *Available*. Although the source code is provided, the accompanying documentation

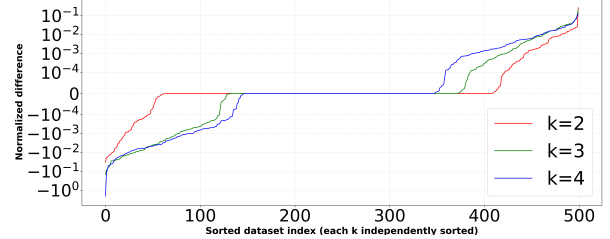


Fig. 10: Quality results difference for GHYPART execution compared with GHYPART-B execution on RTX 4090. Comparison is made on the normalized difference of the partition edge cut cardinality.

and implementation were insufficient to reproduce the complete set of experimental results without substantial and non-trivial additional effort. A more structured organization of the scripts, together with comprehensive documentation of the steps required to reproduce the figures presented in the paper, would have made the software artifact eligible for the *Artifacts Evaluated – Functional* badge.

Nonetheless, the experiments themselves are valid, and therefore the artifact can also be considered to meet the criteria for the *Results Reproduced* badge.

REFERENCES

- [1] Z. Wu, H. Zhao, H. Liu, W. Wen, and J. Li, “ghypart: Gpu-friendly end-to-end hypergraph partitioner,” *ACM Transactions on Architecture and Code Optimization*, vol. 22, no. 1, pp. 1–25, 2025.
- [2] “gHyPart: GPU-friendly End-to-End Hypergraph Partitioner,” https://github.com/zlwu92/gHyPart_TACO, GitHub, 2025, accessed: 2025-11-27.
- [3] “Artifact Review and Badging — Current,” <https://www.acm.org/publications/policies/artifact-review-and-badging-current>, ACM, 2020, accessed: 2025-11-27.
- [4] W. L. Lee, D.-L. Lin, C.-H. Chiu, U. Schlichtmann, and T.-W. Huang, “Hyperg: Multilevel gpu-accelerated k-way hypergraph partitioner,” in *Proceedings of the 30th Asia and South Pacific Design Automation Conference*, ser. ASPDAC ’25. New York, NY, USA: Association for Computing Machinery, 2025, p. 1031–1040. [Online]. Available: <https://doi.org/10.1145/3658617.3697551>
- [5] C. J. Alpert, “The ispd98 circuit benchmark suite,” in *Proceedings of the 1998 International Symposium on Physical Design*, ser. ISPD ’98. New York, NY, USA: Association for Computing Machinery, 1998, p. 80–85. [Online]. Available: <https://doi.org/10.1145/274535.274546>